Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement #07 — Hostel Maintenance & Issue Ticketing System

Name: Aakash Agarwal | SRN: PES2UG24CS009 | Section: A (CSE)

1. Jira Backlog — Epics, User Stories & Story Points

The Lab 1 functional requirements were grouped into 5 Epics, and translated into 8 User Stories, each prioritized and estimated using Fibonacci story points. Total backlog estimate: 45 story points.

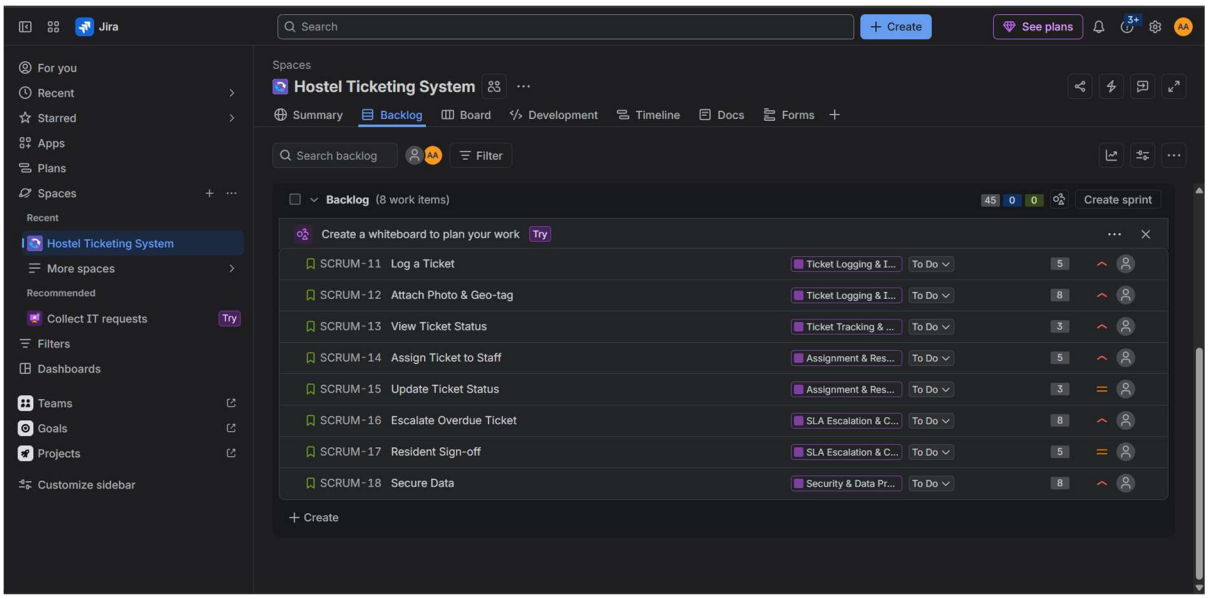


Fig 1. Jira Backlog — Epics, User Stories, priorities, and story points

Epic	User Story	Description	Points
Ticket Logging & Intake	Log a Ticket	Resident logs ticket with room, category, severity	5
Ticket Logging & Intake	Attach Photo & Geo-tag	Auto geo-tagged/timestamped photo attachment	8
Ticket Tracking & Visibility	View Ticket Status	Resident views real-time ticket status	3
Assignment & Resolution Workflow	Assign Ticket to Staff	Warden assigns ticket by category	5
Assignment & Resolution Workflow	Update Ticket Status	Warden updates status with timestamp/remarks	3
SLA Escalation & Closure	Escalate Overdue Ticket	Auto-alert on 24h SLA breach	8
SLA Escalation & Closure	Resident Sign-off	Resident confirms closure / reopens within 48h	5
Security & Data Protection	Secure Data	Encrypt resident data & photos, RBAC	8

2. Sprint Execution

Sprint 1 (30 Aug – 6 Sep 2026) — 21 story points

Stories: Log a Ticket (5), Attach Photo & Geo-tag (8), View Ticket Status (3), Assign Ticket to Staff (5).

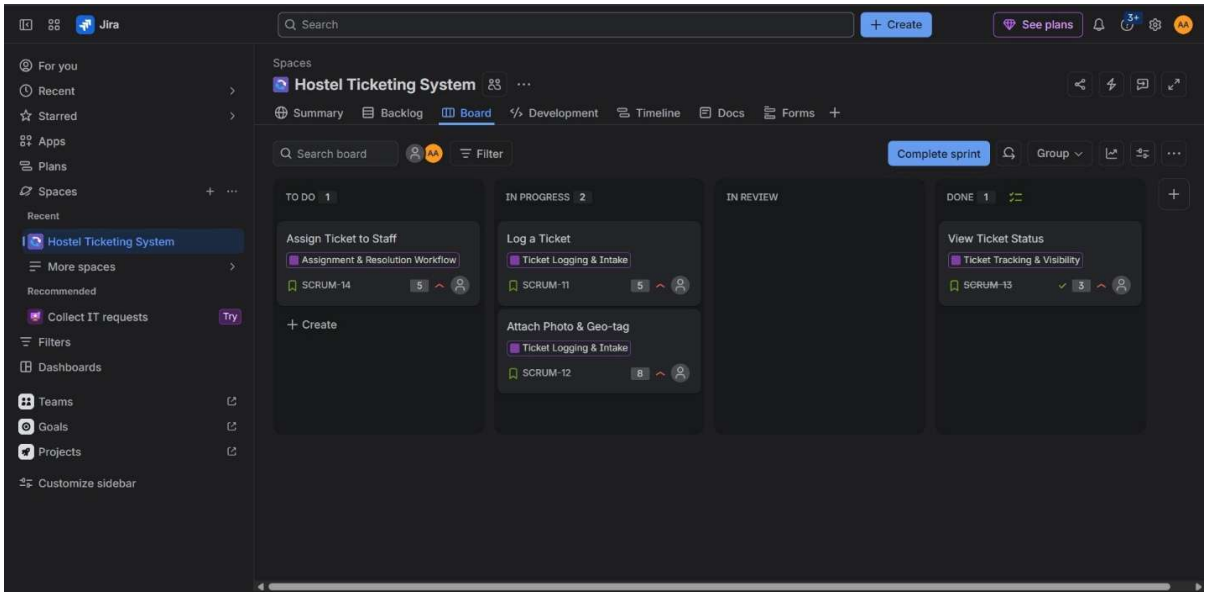


Fig 2. Sprint 1 — Active Sprint board

Sprint 2 (6 Sep – 13 Sep 2026) — 24 story points

Stories: Update Ticket Status (3), Escalate Overdue Ticket (8), Resident Sign-off (5), Secure Data (8).

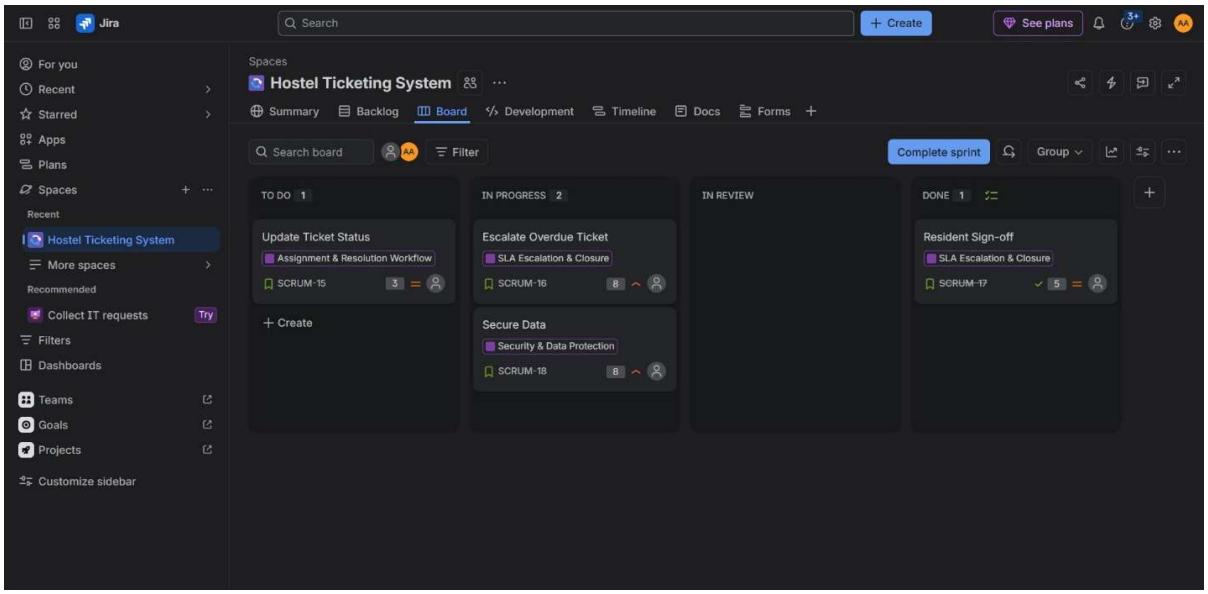


Fig 3. Sprint 2 — Active Sprint board

3. Burndown Charts

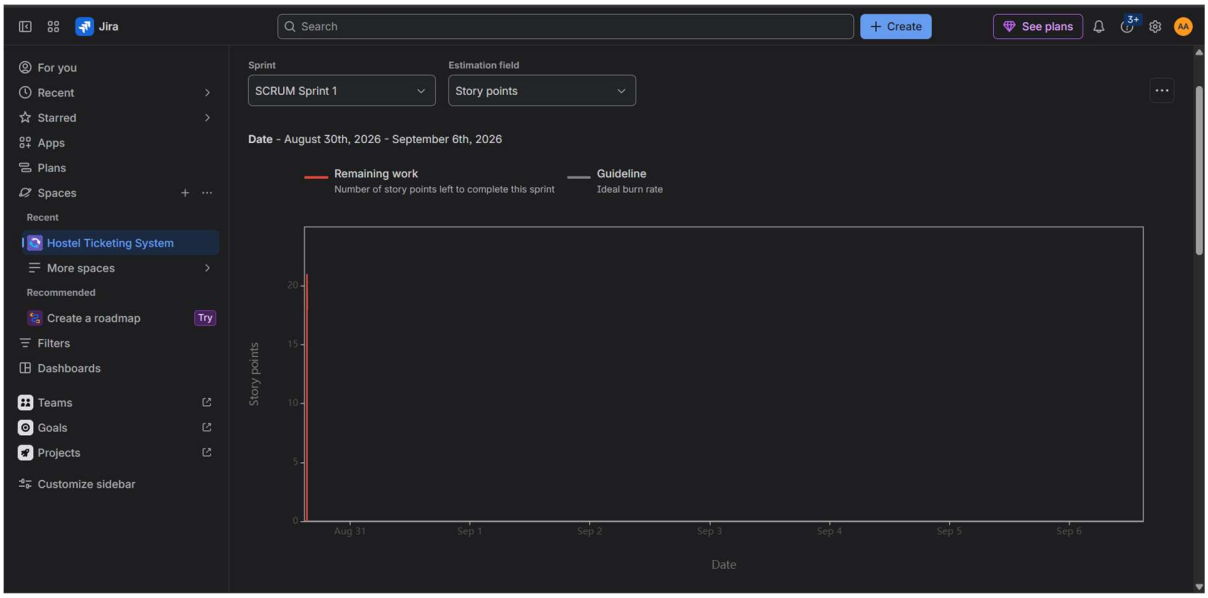


Fig 4. Sprint 1 Burndown Chart



Fig 5. Sprint 2 Burndown Chart

4. Reflection Questions

Q1. Did your estimations reflect the actual effort?

Since this was a simulated sprint rather than real development, actual implementation effort wasn't directly tracked. However, the relative sizing followed a consistent logic: simpler CRUD-style features (e.g., Update Ticket Status, View Ticket Status) were sized at 3 points, moderately complex features involving routing or confirmation logic (Log a Ticket, Assign Ticket to Staff, Resident Sign-off) at 5, and features with higher technical complexity — device integrations, timers, and encryption (Attach Photo & Geo-tag, Escalate Overdue Ticket, Secure Data) — at 8. This scaling is a reasonable proxy for real effort, though it would need to be recalibrated against actual velocity once real development begins.

Q2. Was your backlog well-prioritized?

Yes. Priorities were carried over from the Lab 1 requirements table, where core intake, tracking, assignment, escalation, and security functions were marked High priority, while secondary workflow refinements (status update remarks, resident sign-

off) were Medium. This ensured Sprint 1 delivered the essential path — a resident can log a ticket and see its status, and a warden can assign it — before Sprint 2 layered on escalation, closure, and security hardening.

Q3. How did your simulated sprint align with your plan?

Both sprints completed close to their planned scope. Sprint 1 (21 points) ended with 3 of 4 stories in progress or done and 1 rolled back to the backlog; Sprint 2 (24 points) followed the same pattern. This mirrors a realistic, slightly over-committed sprint — a common and expected outcome in real Agile teams, and a useful signal to plan slightly lighter sprints going forward.

Q4. What insights did the burndown chart give about your team's capacity?

Because the sprint activity was simulated within a single sitting rather than spread across a real week, both burndown charts show the 'remaining work' line dropping almost immediately near the start date rather than declining gradually alongside the grey guideline. In a real sprint, work should be logged incrementally as it happens, producing a burndown line that tracks close to the ideal guideline day-by-day. This simulation highlights that a burndown chart is only a meaningful capacity signal when status updates reflect real, distributed progress — a compressed, single-session simulation cannot show whether a team's pace is actually sustainable.